Multiple URLs for a Single Resource in ASP.NET Core Web API

Sep 24, 2017 by Sam Allen and Dot Net Perls

Single Resource with Multiple URLs in ASP.NET Core Web API

In this article, I will discuss how to set up **Multiple URLs for a Single Resource in the ASP.NET Core Web API** in Visual Studio. These are concepts I will be discussing working with Visual Basic and Visual Studio ASP.NET Core Web API.

In REST's HTTP design, routing flexibility is essential for supporting different client needs. Sometimes, different consumers (e.g. internal tools, mobile apps, or third-party integrations) expect different endpoint names, even if they all serve the same underlying data.

Example Use Cases:

- /api/employee/{id} — used by current dashboard
- /api/employees/{id} — used by a mobile app
- /api/Employees/{id} — used by a legacy systems

ASP.NET Core provides a powerful attribute routing system that allows developers to define multiple routes for a single resource simply by decorating multiple [HttpGet], [HttpPost], etc. attributes on the same method. This flexibility supports:

- **Accurate Code Organization**: Keep the same code base for multiple routes
- **Backward Compatibility**: Support old URLs while introducing new ones
- **REST-Friendly or Legacy-Compatible URLs**: Serve both modern and legacy URLs
- **Different client preferences**: Mobile, web, and partner systems may prefer different naming conventions

How to Accomplish Multiple Resources with Multiple URLs?

A single resource (like an employee) can be accessible through multiple URLs without duplicating the code or creating separate controllers. This approach is:

- Efficient, avoiding redundant, non-concentrated routing
- Scalable, supporting new clients and **multiple route attributes**

Each attribute defines an alternative URL pattern for accessing the same underlying logic. Let's proceed and understand how to create a single resource with multiple URLs using ASP.NET Core Web API in Visual Studio.

Define the Employee Model

Create a file named **Employee.cs** and add the **Models** folder and then copy and paste the following code. The Employee model code is the data blueprint for your API. It defines the properties (ID, Name, Gender, Department, City) that represent an employee entity. This is the core data entity that our Web API will expose to clients.

```
namespace MultipleURLsForSingleResourceInWebAPI.Models
{
    public class Employee
    {
        public int EmployeeID { get; set; }
        public string EmployeeName { get; set; }
        public string Gender { get; set; }
        public int DepartmentID { get; set; }
        public string City { get; set; }
        public static List<Employee> Employees = new List<Employee>
        {
            new Employee { EmployeeID = 1, EmployeeName = "Jennifer", Gender = "Female", DepartmentID = 1, City = "Chicago" },
            new Employee { EmployeeID = 2, EmployeeName = "Sam", Gender = "Male", DepartmentID = 2, City = "Chicago" },
            new Employee { EmployeeID = 3, EmployeeName = "Charlie", Gender = "Male", DepartmentID = 3, City = "Chicago" }
        };
        public static Employee[] GetEmployees()
        {
            return Employees.ToArray();
        }
        public static Employee GetEmployee(int id)
        {
            return Employees.FirstOrDefault(e => e.EmployeeID == id);
        }
    }
}
```

Initialize In-Memory Data

In real scenarios, we would typically use a database or a service connecting to a database. In this article, I create a static list named **EmployeeList** within the Models folder and then copy and paste the following code. Instead of connecting to a database, we initialize a static list with sample data. This keeps the demo focused on routing rather than data access.

```
namespace MultipleURLsForSingleResourceInWebAPI.Models
{
    public static class MockDataSource
    {
        public static List<Employee> EmployeeList = new List<Employee>
        {
            new Employee { EmployeeID = 1, EmployeeName = "Jennifer", Gender = "Female", DepartmentID = 1, City = "Chicago" },
            new Employee { EmployeeID = 2, EmployeeName = "Sam", Gender = "Male", DepartmentID = 2, City = "Chicago" },
            new Employee { EmployeeID = 3, EmployeeName = "Charlie", Gender = "Male", DepartmentID = 3, City = "Chicago" }
        };
        public static int[] GetDepartmentIDs(int[] ids)
        {
            return ids ?? new int[0];
        }
        public static int[] GetDepartmentIDs(int id)
        {
            return new int[] { id };
        }
        public static int[] GetDepartmentIDs()
        {
            return new int[0];
        }
        public static int[] GetDepartmentIDs(int[] ids, int id)
        {
            var result = (ids ?? new int[0]).ToArray();
            if (id > 0)
            {
                var length = result.Length;
                Array.Resize(ref result, length + 1);
                result[length] = id;
            }
            return result;
        }
    }
}
```

Implementing Multiple Routes for a Single Resource in ASP.NET Core Web API

Now, we will create an EmployeesController that exposes endpoints to retrieve employee data. We will configure multiple routes for accessing the same resource. Suppose we have the following resource URL in Postman Collection.

```
http://localhost:5000/api/Employees/GetEmployee?id=1
```

Define the Controller with Multiple Routes in ASP.NET Core Web API

Now, we will create an EmployeesController that exposes endpoints to retrieve employee data. We will configure multiple routes for accessing the same resource. Suppose we have the following resource URL in Postman Collection.

```
[ApiController]
[Route("api/[controller]")]
public class EmployeesController : ControllerBase
{
    [HttpGet("GetEmployee")]
    [HttpGet("GetEmployee/{id:int}")]
    [HttpGet("Employee/{id:int}")]
    [HttpGet("Employee")]
    [HttpGet("Employees")]
    [HttpGet("Employees/{id:int}")]
    [HttpGet("Employees/{ids:int[]}")]
    [HttpGet("Employees/{ids:int[]}/{id:int}")]
    [HttpGet("Employees/{id:int}/{ids:int[]}")]
    [HttpGet("Employees/Departments/{ids:int[]}")]
    [HttpGet("Employees/Departments/{id:int}")]
    [HttpGet("Employees/Departments")]
    [HttpGet("Employees/Departments/{ids:int[]}/{id:int}")]
    [HttpGet("Employees/Departments/{id:int}/{ids:int[]}")]
    public IEnumerable<Employee> GetEmployees(int[] ids, int id)
    {
        var deptIDs = MockDataSource.GetDepartmentIDs(ids, id);
        var result = MockDataSource.EmployeeList.AsEnumerable();
        if (deptIDs.Length > 0)
        {
            result = result.Where(e => deptIDs.Contains(e.DepartmentID));
        }
        if (id > 0)
        {
            result = result.Where(e => e.EmployeeID == id);
        }
        return result.ToArray();
    }
}
```

Code Explanation:

This implementation defines a decorated with [ApiController] attribute, exact specifying a different route.

- **/api/Employees/GetEmployee**
- **/api/Employees/GetEmployee/{id:int}**
- **/api/Employees/Employee/{id:int}**

This configuration allows clients to access the same resource using any of these URLs. Now, run the application and access the above resources using the different URLs and you should get the same result as shown in the image below.

Using a web browser or Postman, accessing any of the URLs should display the same list of employees, confirming that the multiple routes successfully access the same underlying resource. Now, let's add a new route to the controller.

What Happens if Two Action Methods Use the Same Route in ASP.NET Core Web API?

Assigning the same route to multiple action methods will lead to conflicts, and routing becomes ambiguous. ASP.NET Core will throw an error when two endpoints share the same route, because it can't determine which one should handle the request. To avoid conflicts in Web APIs, ensure each route is unique. Now, you can see how two actions refer to the EmployeesController with the same route, i.e., /api/Employees/All.

```
[ApiController]
[Route("api/[controller]")]
public class EmployeesController : ControllerBase
{
    // Find Action Method
    [HttpGet("Employees/All")]
    public IEnumerable<Employee> GetAllEmployees(int[] ids)
    {
        var deptIDs = MockDataSource.GetDepartmentIDs(ids);
        var result = MockDataSource.EmployeeList.AsEnumerable();
        if (deptIDs.Length > 0)
        {
            result = result.Where(e => deptIDs.Contains(e.DepartmentID));
        }
        return result.ToArray();
    }
    // Find Action Method
    [HttpGet("Employees/All")]
    public IEnumerable<Employee> GetAllEmployees(int id)
    {
        var result = MockDataSource.EmployeeList.AsEnumerable();
        if (id > 0)
        {
            result = result.Where(e => e.EmployeeID == id);
        }
        return result.ToArray();
    }
}
```

Now, when the URL is hit, you'll get the getting a warning as shown in the image below.

GET
http://localhost:5000/api/Employees/All
Response (500)
An unhandled exception occurred while processing the request.
AmbiguousMatchException: The request matchesmultiple endpoints. Matchingendpoints: MultipleURLsForSingleResourceInWebAPI.Controllers.EmployeesController.GetAllEmployees (